

Plan 001: Corrigir testes de UI (Streamlit AppTest)

Executor instructions: Follow this plan step by step. Run every verification command and confirm the expected result before moving to the next step. If anything in the “STOP conditions” section occurs, stop and report — do not improvise. When done, update the status row for this plan in `plans/README.md`.

Drift check (run first): `git diff --stat 94369a2..HEAD -- tests/ui/ src/ui/` Se qualquer arquivo in-scope mudou desde `94369a2`, compare os excertos de “Current state” antes de prosseguir; se houver divergência, PARE.

Status

- **Priority:** P1
- **Effort:** M
- **Risk:** LOW
- **Depends on:** none
- **Category:** tests
- **Planned at:** commit `94369a2`, 2026-06-28

Why this matters

3/5 testes de UI falham. Sem um baseline de testes de UI verde, qualquer regressão visual ou funcional passa despercebida. O `AppTest` do Streamlit permite testar a UI sem browser real, e os dois testes que passam (`test_plot_multiaxis_returns_figure`, `test_plot_multiaxis_empty`) provam que o mecanismo funciona — o problema está nos testes que chamam `app.run()`.

Current state

- `tests/ui/test_streamlit_app.py` — 2 testes, ambos falham com timeout (`RuntimeError: AppTest script run timed out after 10(s)`)
- `tests/ui/test_streamlit_e2e.py` — 3 testes, o e2e falha com `AssertionError: assert 'Serie dinamica' in []` (subheaders vazios)

Falha observada:

```
# test_streamlit_app.py
RuntimeError: AppTest script run timed out after 10(s)
→ missing ScriptRunContext no stderr

# test_streamlit_e2e.py
assert 'Serie dinamica' in [] → subheaders vazio após monkeypatch
```

Testes que passam (`tests/unit/test_services.py`): usam `_FakeRepo` com dados sintéticos e chamam serviços diretamente, sem Streamlit.

Commands you will need

Purpose	Command	Expected on success
Tests unit	<code>python -m pytest tests/unit -q</code>	12 passed
Tests UI	<code>python -m pytest tests/ui -q</code>	5 passed
All tests	<code>python -m pytest tests/ -q</code>	all pass

Scope

In scope (the only files you should modify):

- `tests/ui/test_streamlit_app.py`
- `tests/ui/test_streamlit_e2e.py`

Out of scope (do NOT touch):

- `src/ui/streamlit_app.py` — o código da UI não tem bugs conhecidos; os testes é que precisam se adaptar ao comportamento real do AppTest.
- `tests/unit/` — estão verdes.

Git workflow

- Branch: `advisor/001-fix-ui-tests`
- Commit message style: conventional commits (`fix: corrige testes de UI...`)
- Do NOT push ou abrir PR.

Steps

Step 1: Diagnosticar timeout do AppTest

O `test_ui_initial_render_has_core_controls` chama `app.run(timeout=10)` e o AppTest falha com timeout + `missing ScriptRunContext`.

O `ScriptRunContext` é gerido pelo Streamlit internamente — o warning no stderr é esperado em modo de teste. O timeout pode ser insuficiente ou o `app.py` estar tentando conectar em SQL Server (inexistente).

Leia `app.py` e `src/ui/streamlit_app.py` para confirmar: `app.py` chama `main()`, que chama `_build_services()` → `SqlServerConnection(SETTINGS)`. **Sem SQL Server disponível,**

`_build_services()` lança exceção.

Solução: monkeypatch `_build_services` nos testes de UI (igual o `test_streamlit_e2e.py` já faz com `monkeypatch.setattr`).

Verifique: `python -m pytest`

```
tests/ui/test_streamlit_app.py::test_ui_initial_render_has_core_controls -q 2>&1 | tail -5
```

→ Ainda falha com timeout (baseline).

Step 2: Aplicar monkeypatch no test_streamlit_app.py

Adicione o mesmo padrão de monkeypatch usado em `test_streamlit_e2e.py` .

No topo de `tests/ui/test_streamlit_app.py` , importe os fakes já existentes ou crie fakes inline:

```
from tests.ui.test_streamlit_e2e import _FakeTestService, _FakeObservationService, _FakeExportService, _FakeDiagnosticsService
import src.ui.streamlit_app as streamlit_app
```

Adicione o fixture `monkeypatch` do pytest como primeiro parâmetro de cada teste que chama `app.run()` , e antes de `app.run()` :

```
def test_ui_initial_render_has_core_controls(monkeypatch) -> None:
    monkeypatch.setattr(
        streamlit_app,
        "_build_services",
        lambda: (_FakeTestService(), _FakeObservationService(), _FakeExportService(), _FakeDiagnosticsService()),
    )
    app = AppTest.from_file("app.py")
    app.run(timeout=15)
    # ... asserts existentes ...
```

Ajuste `timeout` de 10 para 15 segundos.

Verifique: `python -m pytest tests/ui/test_streamlit_app.py -q` → 2 passed

Step 3: Corrigir test_ui_initial_state_shows_empty_message

O teste busca "Sem dados carregados" em `app.info` , mas a UI exibe "Nenhum ensaio carregado" (texto real em `streamlit_app.py`).

Mude o assert:

```
# Antes:
assert any("Sem dados carregados" in value for value in info_values)
# Depois:
assert any("Nenhum ensaio carregado" in value for value in info_values)
```

Verifique: `python -m pytest tests/ui/test_streamlit_app.py -q` → 2 passed

Step 4: Diagnosticar test_ui_e2e_load_flow_and_exports

O teste usa `monkeypatch.setattr(streamlit_app, "_build_services", ...)` mas falha com `assert 'Serie dinamica' in []`.

Isso indica que após clicar em “Carregar ensaio”, os subheaders esperados não foram renderizados. Pode ser que o fluxo condicional na UI (`if rows is None or rows.empty`) esteja avaliando de forma diferente com o fake service.

Leia `streamlit_app.py` entre as linhas 250-280 para ver o que é renderizado após o carregamento. Ajuste o assert do teste para corresponder ao texto exato renderizado (ex: "Operação" como label da tab, não "Serie dinamica").

Verifique: `python -m pytest tests/ui/test_streamlit_e2e.py -q` → 3 passed

Step 5: Rodar suite completa de UI

Verifique: `python -m pytest tests/ui -q` → 5 passed in < 60s

Test plan

- Nenhum teste novo — apenas correção dos existentes.
- Manter os 2 testes unitários que já passam (`test_plot_multiaxis*`).
- `test_ui_e2e_load_flow_and_exports` deve continuar usando monkeypatch (não requer SQL Server real).

Done criteria

- ☐ [] `python -m pytest tests/ui -q` → 5 passed
- ☐ [] `python -m pytest tests/unit -q` → 12 passed (nada quebrado)
- ☐ [] Nenhum arquivo fora de `tests/ui/` foi modificado

STOP conditions

- Se `app.run(timeout=15)` continuar falhando, reporte o stacktrace completo.
- Se o teste e2e exigir modificação em `src/ui/streamlit_app.py`, PARE e reporte — o plano prevê só mexer nos testes.

Maintenance notes

- Se novos componentes forem adicionados à UI, os fakes em `test_streamlit_e2e.py` precisarão ser estendidos.
- O monkeypatch de `_build_services` é frágil se a função mudar de nome. Uma alternativa futura: injetar serviços via parâmetro ou `st.session_state`.